SUBPROGRAMS

General Characteristics of Subprograms

- A subprogram has a single entry point.
- The calling program is suspended during execution of the called subprogram.
- Control always returns to the caller when the subprogram execution terminates.

Subprogram Definition

A subprogram definition describes the interface to and the actions of the subprogram abstraction.

A subprogram call is the explicit request that the subprogram be executed.

Subprogram Definition

A subprogram header

- Specifies that the following syntactic unit is a subprogram definition. This is usually with a special word.
- Provides a name for the subprogram.
- May optionally specify a list of parameters.

Subprogram Definition

Example: header

procedure *SUB*(*parameters*)

An example of a subprogram header in Ada

This subprogram definition uses a special word (**procedure**).

Subprogram Definition

Example: header

void *SUB*(*parameters*)

An example of a subprogram header in C

This subprogram definition does not use any special word.

Parameters

A subprogram can gain access to data (to be processed) in two ways,

- Direct access to nonlocal variables visible to the subprogram
- Parameter passing

Parameters

Formal Parameters vs Actual Parameters

Formal parameters are the parameters in the subprogram header.

These parameters are placeholders of the data passed to the subprogram.

Parameters

Formal Parameters vs Actual Parameters

Actual parameters are the parameters specified in a subprogram call.

These parameters are to be bound to the formal parameters of the subprogram.

Parameters

Formal parameters and actual parameters correspondence (binding of actual to formal) can be,

- by position;
- by keyword; or
- combination of both

Parameters

Positional parameters

- First actual parameter is bound to the first formal parameter, …
- Used by most programming languages
- Good for relatively short parameter lists.

How about for long parameter lists?

Parameters

Keyword parameters

- Formal parameter to which the actual parameter is to be bound is specified with the actual parameter
- Can appear in any order in the actual parameter list.

There is a need to know the names of the formal parameters in this method.

Parameters

Keyword parameters

Example,

```
SUM(VAL => MY_VAL,
        RATE => MY_RATE);
```

For this example, VAL and RATE are the formal parameters of SUM

Parameters

Positional and keyword parameters combination

Example,

```
SUM(MY_VAR,
        THIRD => MY_VAL,
        SECOND => MY_RATE);
```

*Only restriction with this is that once a keyword parameter appears in the actual parameter list, all remaining parameters must be keyworded. WHY?

Parameters

In C++, FORTRAN 90, and Ada, formal parameters can have default values.

A default is used if no actual parameter is passed to the formal parameter.

This makes it possible to have fewer actual parameters in a call than formal parameters in the subprogram definition.

Parameters

C++ formal parameters having default values

Example,

```
void display(int w, int x, int y=2, int z=3) {
        printf("%d %d %d %d", w, x, y, z);
}
…
```

function call: ***display*(4, 3);**

Parameters

C++ formal parameters having default values

The default parameters must appear last because parameters are positional.

For formal parameters having no default values, they are subject to positional association with the actual parameters.

Parameter Passing

Semantic models of parameter passing

- in mode
- out mode
- inout mode

Parameter Passing

Implementation models of parameter passing

- Pass-by-Value (in mode)
- Pass-by-Result (out mode)
- Pass-by-Value-Result (inout mode)
- Pass-by-Reference (inout mode)
- Pass-by-Name (inout mode)

References

- Sebesta, R. W. (????). Concepts of programming languages, fifth edition. Addison Wesley.
- Sebesta, R. W. (1999). Concepts of programming languages, fourth edition. Addison Wesley.